



universität
uulm

Interactive Computer Graphics

Chapter 5 – Clipping

Summer Term 2026

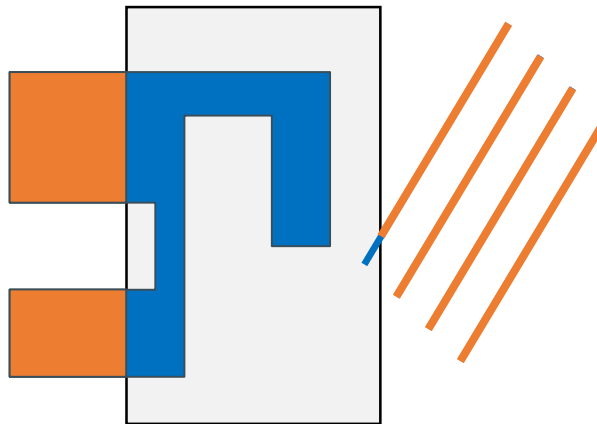
Timo Ropinski



Visual Computing Group
Institute of Media Informatics

Clipping Projected Geometry 1/2

- Now that the geometry has been transformed to its representation on the screen, parts outside screen must be clipped away
 - Clipping ensures that following costly computations (e.g., rasterization, illumination, shading) are only applied to visible parts
- Since clipping is performed for all geometry, algorithms are highly optimized



Clipping Projected Geometry 2/2

- Analytical clipping
 - Clipping before raster conversion, clipped primitives are raster-converted
 - Compute time savings if clipped primitive is much smaller than unclipped
 - OpenGL: clipping against canonical view volume
- Pixel-based clipping
 - Clipping during or after raster conversion
 - Only pixels in the clip region are used
 - Works with any given clip regions
 - OpenGL: possible with fragment shaders, stencil buffer or scissoring

Chapter Outline

5.1 Cohen-Sutherland Line Clipping

5.2 Cyrus-Beck Line Clipping

5.3 Sutherland-Hodgman Polygon Clipping

5.4 Weiler-Atherton Polygon Clipping

5.5 Greiner-Hormann Polygon Clipping

References



universität
uulm

5.1 Cohen-Sutherland Line Clipping

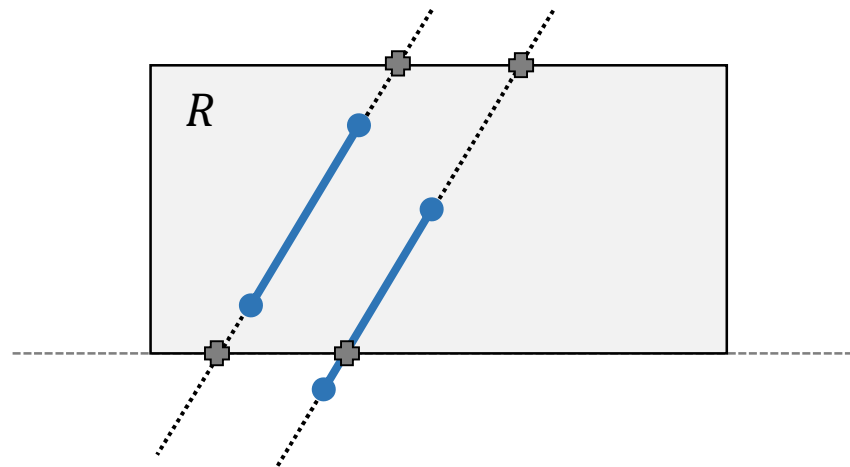
Clip lines against rectangle



Visual Computing Group
Institute of Media Informatics

Brute-Force Line Clipping 1/2

- Check each edge of the rectangle for intersection with line
 - Intersection calculation for the line defined by the edge, and the line
 - If intersection lies within edge and within line, clipping necessary

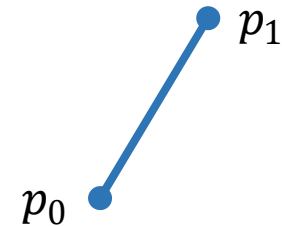


Brute-Force Line Clipping 2/2

- Using the parameter representation for lines

$$p(t) = p_0 + t \cdot (p_1 - p_0) = \begin{pmatrix} x_0 \\ y_0 \end{pmatrix} + t \cdot \begin{pmatrix} x_1 - x_0 \\ y_1 - y_0 \end{pmatrix},$$

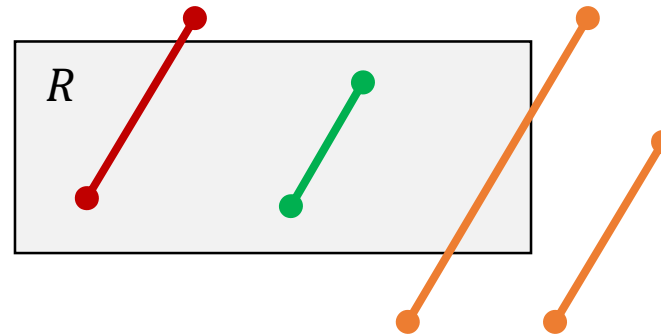
whereby line segment s is defined as all points
 $\{p(t): 0 \leq t \leq 1\}$



- Intersection calculation
 - Parameter representation for s and for an edge e of R
 - Calculation of the intersection given by t_e and t_s
 - $0 \leq t_e \leq 1$ and $0 \leq t_s \leq 1 \Rightarrow$ intersection lies in both segments and is therefore clip intersection
- Disadvantage: considerable computational effort, inefficient in practice

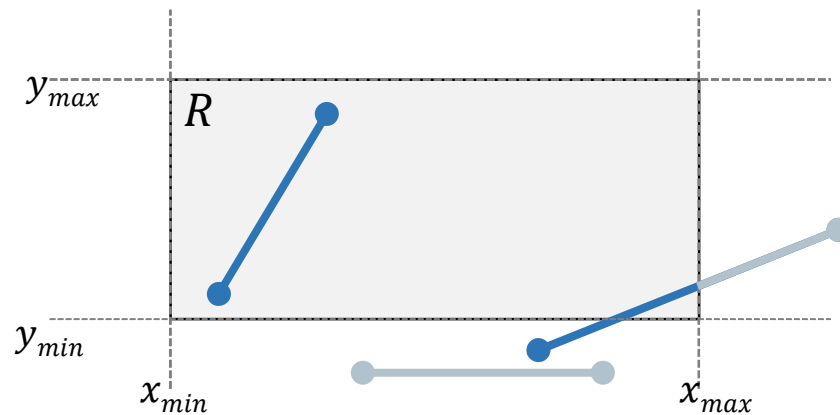
Endpoint Analysis 1/2

- Idea: Use end point analysis to create new lines
 - Clipping against a rectangle yields a line
 - Clipping against a convex polygon results in a line
- Endpoint analysis
 - Line segment $s = (p_0, p_1)$, $p_0 = (x_0, y_0)$ and $p_1 = (x_1, y_1)$



Endpoint Analysis 2/2

- Clipping lines against a clip rectangle by reduction to endpoint analysis



Binary Coding of Half-Planes 1/3

- Classification of the plane in nine regions
 - Classify a point by one bit each for belonging to each of the 4 half-planes

1001	1000	1010
0001	0000	0010
0101	0100	0110

Binary Coding of Half-Planes 2/3

- Endpoint classification
 - Line segment $s = (p_0, p_1)$
 - Outcodes:
 - $oc0 = outcode(p_0)$, $oc1 = outcode(p_1)$

1001	1000	1010
0001	0000	0010
0101	0100	0110

Binary Coding of Half-Planes 3/3

- Clipping, if $(oc0 \ \& \ oc1) = 0$:
 - Choose an order for clipping (e.g., top, bottom, right, left)
 - Select clip edge that is crossed by the line
 - Check done on basis of bit codes, which **differ in at least 1 bit**
 - Clip s at this clip edge
 - Calculate intersection with clip edge, which fixes either x or y that can be put into the equation
 - Example: Intersection with top edge
 - $p.y = y_{max}$
 - $p.x = x_0 + (x_1 - x_0) \cdot (y_{max} - y_0) / (y_1 - y_0)$
 - Line is divided into two segments, from which one is outside and discarded
 - Iterate with newly generated line

Pseudo Code

```
void cohensutherlandLineClipper(Line s, Rectangle r) {
    int oc0 = outcode(s.p0, r);
    int oc1 = outcode(s.p1, r);
    bool done = false;
    while (!done) {
        if (oc0 == 0 && oc1 == 0) {
            drawLine(s); done = true;
        } else if (oc0 & oc1 != 0) {
            done = true;
        } else {
            if (top edge intersected) {
                calculate intersection with top edge
                replace and reclassify endpoint
            }
            else if (bottom edge intersected) { ... }
            else if (right edge intersected) { ... }
            else if (left edge intersected) { ... }
        }
    }
}
```

```
int outcode(Point p, Rectangle r) {
    int c = 0;
    if (p.y > r.max().y())
        c |= 8;
    else if (p.y < r.min().y())
        c |= 4;
    if (p.x > r.max().x())
        c |= 2;
    else if (p.x < r.min().x())
        c |= 1;
    return c;
}
```

Example

1001

1000

1010

0001

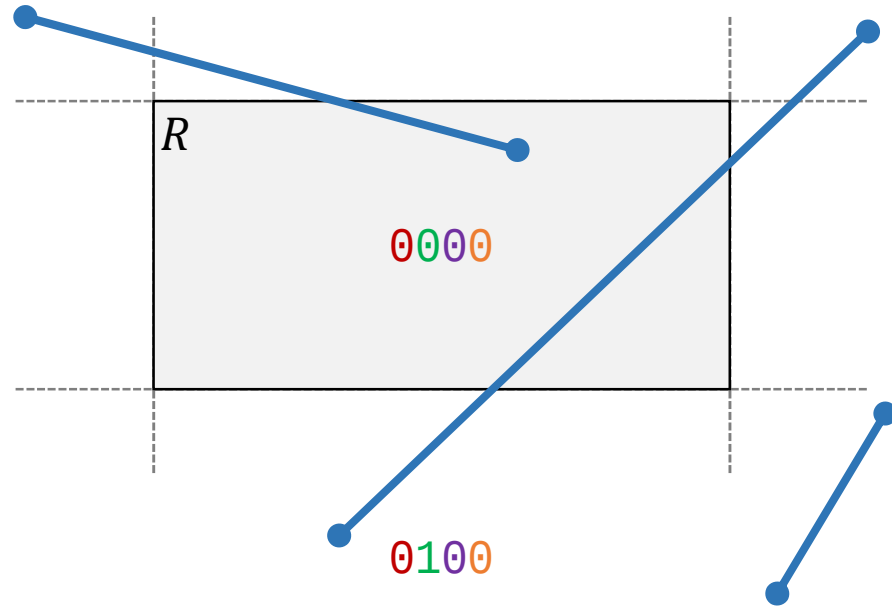
0000

0010

0101

0100

0110



Evaluation

- Occasionally performs several (unnecessary) clip operations on a line as test and clippings are done in a fixed edge order
- Especially efficient when
 - almost all lines are outside
 - almost all lines are inside
- Can be generalized to the three-dimensional case



universität
uulm

5.2 Cyrus-Beck Line Clipping

Clip lines against convex polygon



Visual Computing Group
Institute of Media Informatics

Cyrus-Beck Line Clipping

- Introduced in 1978 by Cyrus and Beck
 - Algorithm for clipping lines against a convex polygon in the plane
 - Expandable to clipping lines against a convex polyhedron in space
 - Fundamentally different approach than Cohen-Sutherland
- Further development in 1984 by Liang and Barsky
 - Optimization of Cyrus-Beck for axis-parallel, rectangular clip regions
- Main idea
 - Exploit parametric representation
 - First calculation of the parameter values of the intersection points, then only calculation of the intersection point coordinates
 - Avoid repeated clip coordinate calculation

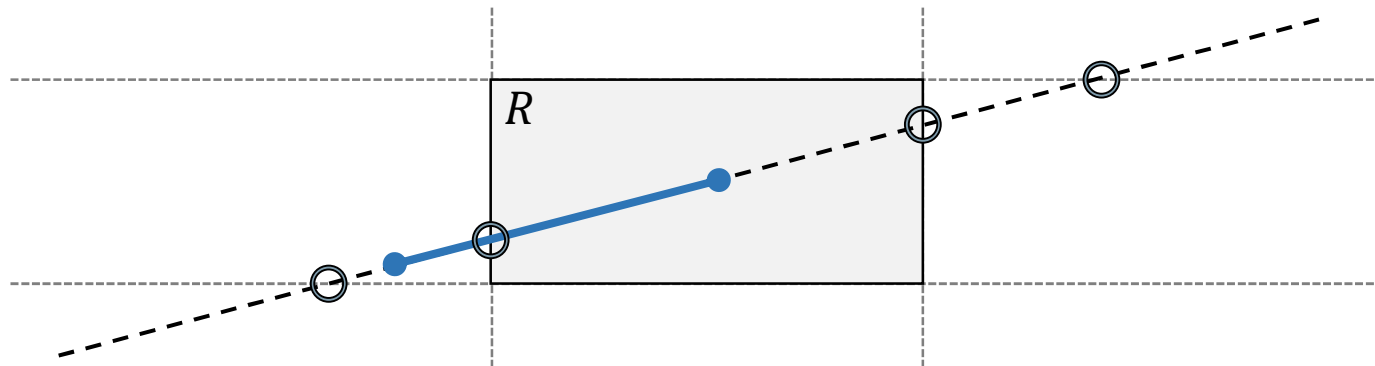
Parametric Line Representation

- Parametric line representation written as follows

$$p(t) = p_0 + t \cdot (p_1 - p_0), 0 \leq t \leq 1$$

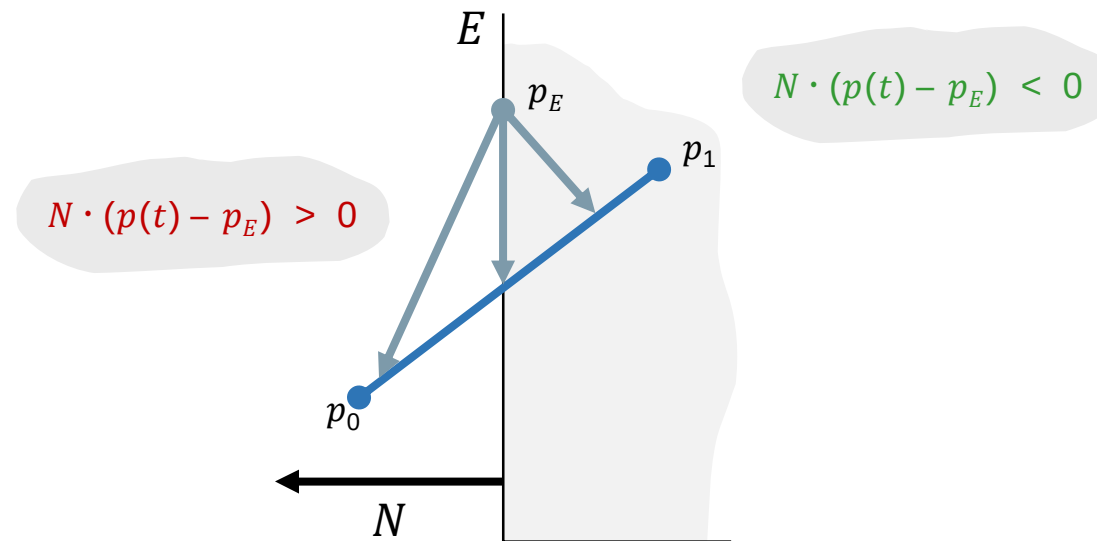
- Proceeding

- Find the 4 t 's for the 4 clip edges
- Decide which of these form clip intersections
- Calculate these intersections (maximum 2)



Intersection Point Calculation 1/2

- Consider edge E and its outward pointing edge normal N
- Through dot product $N \cdot (p(t) - p_E)$ it can be decided in which half-space a line point $p(t)$ lies



Intersection Point Calculation 2/2

- Determine intersection by calculating t for

$$N \cdot (p(t) - p_E) = 0$$

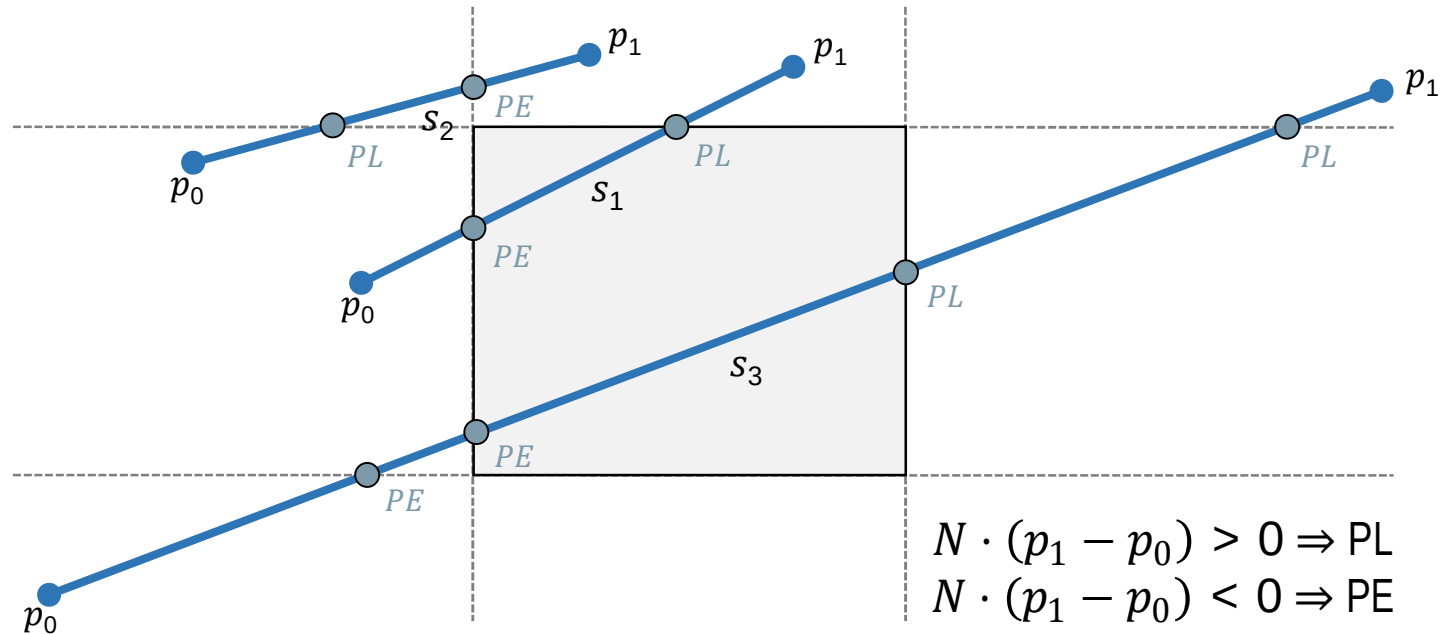
- Substitution of $p(t)$ and regrouping gives

$$t = \frac{N \cdot (p_0 - p_E)}{N \cdot (p_1 - p_0)}$$

- Solution exists only if denominator not zero, i.e., $N \cdot (p_1 - p_0) \neq 0$
 - Line must not be degenerated
 - Line must not be parallel to the edge
- Application for the 4 edges $\Rightarrow$ 4 values for t
- Elimination of t -values that are not in the range $[0, 1]$

Determine Relevant t-Values

- Classification of intersection points as potentially entering (PE) or potentially leaving (PL) with respect to interior of half-plane edge



Pseudo Code

```
void cyrusBeckLineClipper(Line s, Rectangle r) {
    precalculate  $N_i$  for edges of r and select  $P_E$ 
     $D = s.p1 - s.p0$ 
    if (s.p0 == s.p1) {
        degenerated line, clip as point;
    } else {
         $t_E = 0$ ;  $t_L = 1$ ;
        for each candidate intersection with a clip edge {
            if ( $N_i \cdot D \neq 0$ ) { // only for non-parallel lines...
                calculate t;
                use sign of  $N_i \cdot D$  to categorize as PE or PL
                if (PE)  $t_E = \max(t_E, t)$ ;
                else if (PL)  $t_L = \min(t_L, t)$ ;
            }
        }
        if ( $t_E < t_L$ ) { drawline( $P(t_E), P(t_L)$ ); }
        else { ignore line; }
    }
}
```

Axis-Parallel Clipping Region

- When clipping region is axis parallel and given by $[x_{min}, x_{max}] \times [y_{min}, y_{max}]$, following values occur

Clip Edge E	Normal N	p_E	$p_0 - p_E$	$t = \frac{N \cdot (p_0 - p_E)}{N \cdot (p_1 - p_0)}$
$x = x_{min}$	$(-1, 0)$	(x_{min}, y)	$(x_0 - x_{min}, y_0 - y)$	$\frac{-(x_0 - x_{min})}{(x_1 - x_0)}$
$x = x_{max}$	$(1, 0)$	(x_{max}, y)	$(x_0 - x_{max}, y_0 - y)$	$\frac{-(x_0 - x_{max})}{(x_1 - x_0)}$
$y = y_{min}$	$(0, -1)$	(x, y_{min})	$(x_0 - x, y_0 - y_{min})$	$\frac{-(y_0 - y_{min})}{(y_1 - y_0)}$
$y = y_{max}$	$(0, 1)$	(x, y_{max})	$(x_0 - x, y_0 - y_{max})$	$\frac{-(y_0 - y_{max})}{(y_1 - y_0)}$

Comparison with Cohen-Sutherland

- Algorithm of Cohen-Sutherland
 - Sometimes performs useless clip operations, as clipping and testing are done in a fixed order
 - Efficient when outcode tests can be performed inexpensively (e.g., by bitwise operations) and the majority of line segments are trivially accepted or rejected
 - Especially efficient for large clip rectangles, when almost all lines are inside, and small clip rectangles when almost all lines are outside
 - Generalizable to 3D
- Algorithm of Cyrus-Beck / Liang-Barsky
 - Beneficial if many lines really need to be clipped, because calculating the coordinates of the clip intersections is delayed as long as possible and the necessary tests are performed using the parameter values
 - Generalizable to convex clip-polygons and 3D



universität
uulm

5.3 Sutherland-Hodgman Polygon Clipping

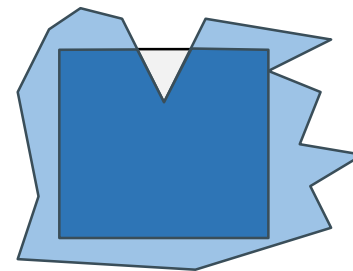
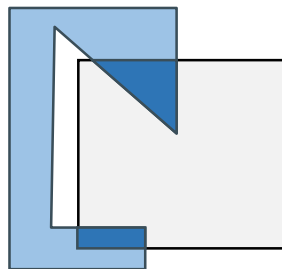
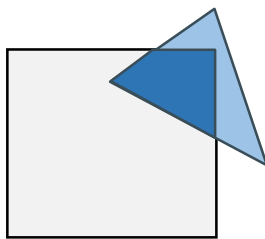
A pipelined version of the scalar product algorithm extended to clip polygons



Visual Computing Group
Institute of Media Informatics

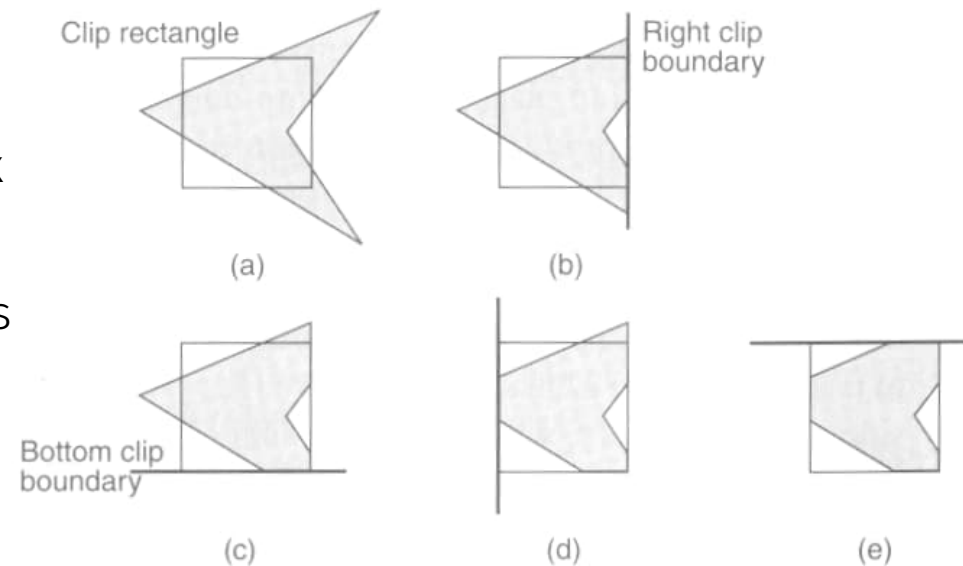
Sutherland-Hodgman Polygon Clipping 1/2

- Background
 - General polygon clipping (arbitrary polygon against convex polygon)
 - Originally developed for visibility detection (apply before rasterization to eliminate non-visible polygon pieces)
- Polygon clipping is more complex than line clipping
 - Clipped polygons must be closed
 - A polygon can be split into several components by clipping



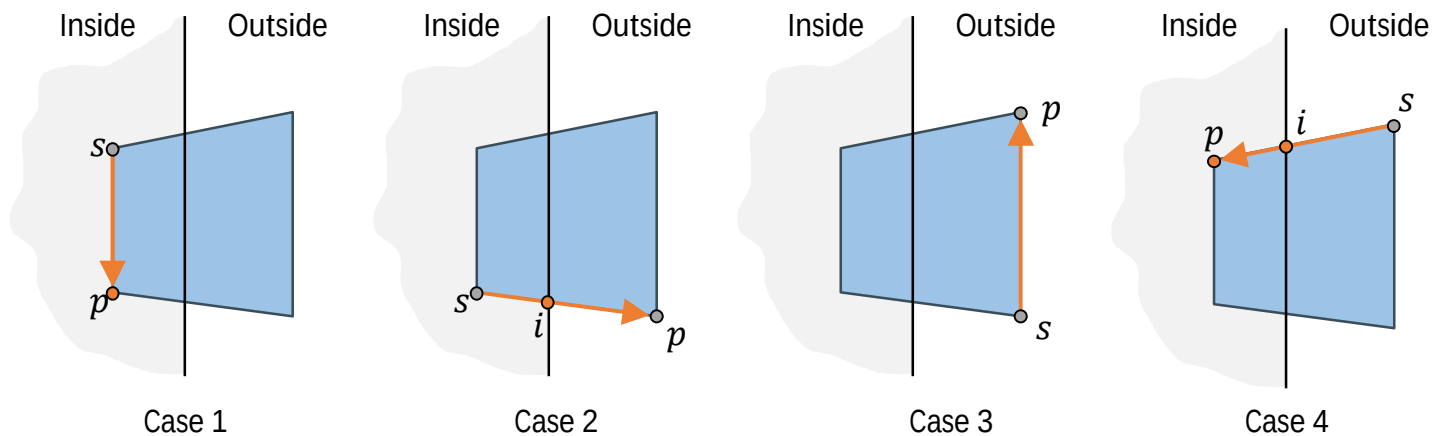
Sutherland-Hodgman Polygon Clipping 2/2

- Iterative algorithm paradigm
 - Clipping a polygon takes place against a half-plane
 - Clipping is performed sequentially for each edge of the clipping region
- Input
 - Convex clipping region
 - Polygon: convex or non-convex
- Output
 - One or several partial polygons inside clip region



Four Cases

- Depending on which case occurs, following points added to output list
 - Case 1: p
 - Case 2: Intersection point i
 - Case 3: --
 - Case 4: Intersection point i and p

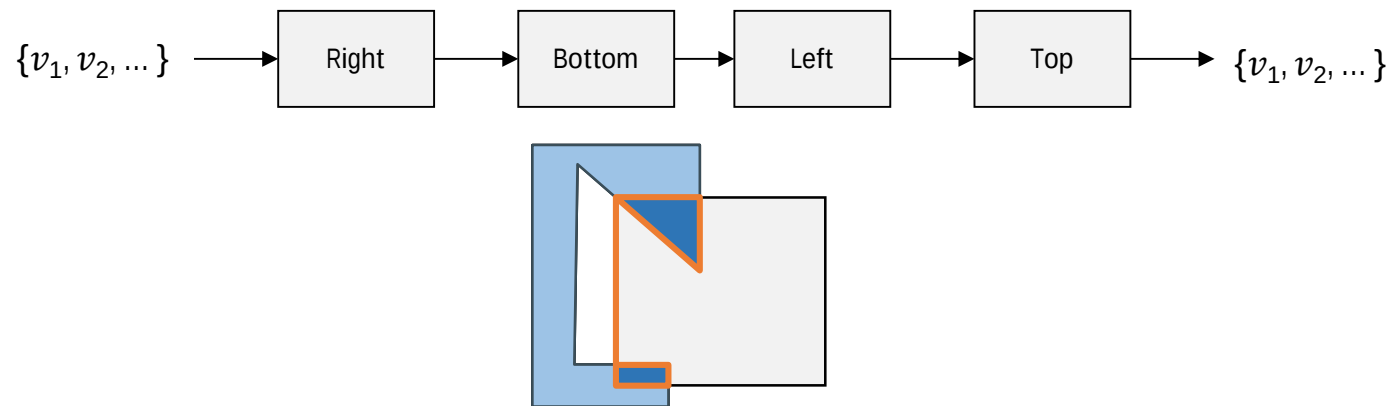


Proceeding

- Polygon to be clipped is defined by
 - Vertices: $[v_1, v_2, \dots, v_n]$
 - Contour: $[v_1, v_2, \dots, v_n, v_1]$
 - Edges: $[v_i, v_{i+1}]$ and $[v_n, v_1]$
- Vertex sequence (contour) is clipped in each pass against the half-plane
- Output of a run is again a sequence of vertices

Pipeline Version

- Speed can be optimized through pipeline version
 - Input: sequence of polygon vertices processed by "Pipeline of Clippers"
- Passing on to the next level once a new vertex has been identified
- No cache is needed during clipping
 - Suitable for implementation by hardware



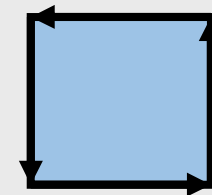
Pseudo Code

```
public VertexList shClipper(VertexList inputVertices, Edge clipE) {
    VertexList outputVertices;           // output list of vertices
    int N = inputVertices.size();        // number of vertices to be processed
    Vertex s = inputVertices.get(N-1);    // get last vertex
    for (int j = 0; j < N; j++) {
        Vertex p = inputVertices.get( j );
        if (inside(p, clipE)) {
            if (inside(s, clipE)) {      // case 1
                outputVertices.add( p );
            } else {                     // case 4
                outputVertices.add( intersect(s, p, clipE) );
                outputVertices.add( p );
            }
        } else if (inside(s, clipE)) {  // case 2
            outputVertices.add( intersect(s, p, clipE) );
        }
        s = p;
    }
    return outputVertices;
}
```

Inside Test

- Inside test for rectangular clip regions

```
boolean inside(Vertex v, Edge e) {           // e is edge of a clip rectangle
    if (e.v1.x > e.v0.x)                       // bottom
        if (v.y >= e.v0.y) return true;
    else if (e.v1.x < e.v0.x)                 // top
        if (v.y <= e.v0.y) return true;
    else if (e.v1.y > e.v0.y)                 // right
        if (v.x <= e.v1.x) return true;
    else if (e.v1.y < e.v0.y)                 // left
        if (v.x >= e.v0.x) return true;
    return false;
}
```



- General: is the point to the left of the directed edge?
 - Check sign of dot product of clip edge normal and point's position vector
 - Direction: from the starting point to the end point
 - Polygon vertices are arranged counterclockwise

Intersection Computation

- For rectangular clip regions

```
Vertex intersect(Vertex p, Vertex s, Edge e) {  
    Vertex i;  
    if (e.v0.y == e.v1.y) {        // horizontal edge  
        i.y = e.v0.y;  
        i.x = p.x + (e.v0.y - p.y) * (s.x - p.x) / (s.y - p.y);  
    } else {                        // vertical edge  
        i.x = e.v0.x;  
        i.y = p.y + (e.v0.x - p.x) * (s.y - p.y) / (s.x - p.x);  
    }  
    return i;  
}
```



universität
uulm

5.4 Weiler-Atherton Polygon Clipping

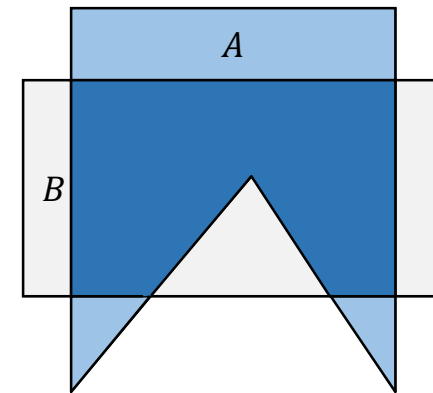
Clip arbitrary polygons against arbitrary clip polygons



Visual Computing Group
Institute of Media Informatics

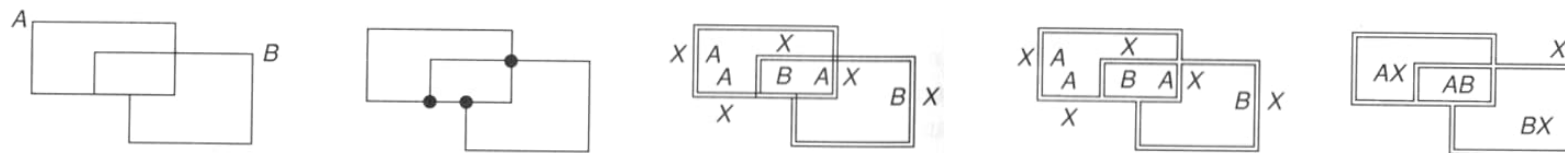
Weiler-Atherton Polygon Clipping

- Basic idea: edges of the clip polygon A and the polygon B to be clipped divide the plane into disjoint regions



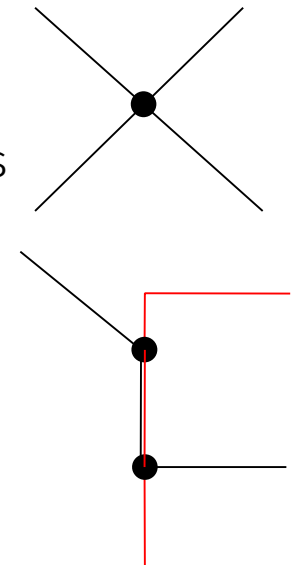
Example Run

- Proceeding best illustrated by example run
- Given case: two partially overlapping polygons



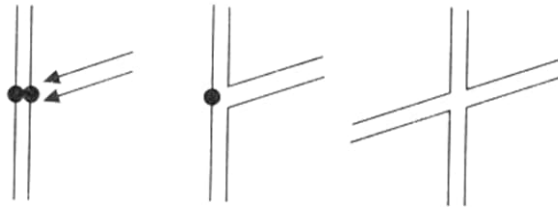
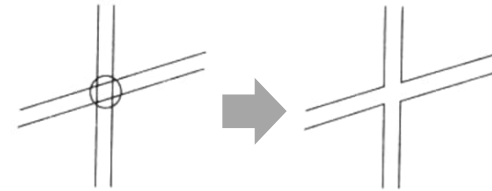
Intersection Types

- There are two intersection types that are distinguished
- Transversal intersections
 - Two edges of the two polygons intersect at one point
 - Procedure: intersection is inserted in vertex list of both polygons
- Segment overlaps (non-transversal)
 - Two edges of polygons overlap completely or partially
 - Procedure: Each corner point on section segment is inserted in the other polygon



Contour Processing

- In order to find the regions-enclosing contours, one must eliminate overlaps of the contours
- Transversal intersections
 - Take the contour edges of A and insert the contour edges of B one after the other
 - Procedure: modify doubly linked list, and use end point comparison to check on which side the contour edges must be inserted



- Different treatment of non-transversal transitions necessary

Evaluation

- Extension to multiple polygons
 - Efficiently possible through iterative application
 - Alternatively, intersection points can be generated for all polygons in the first step
- Conclusion
 - Efficient clipping of arbitrary polygons
 - Role of the polygons is interchangeable
 - Complex implementation!

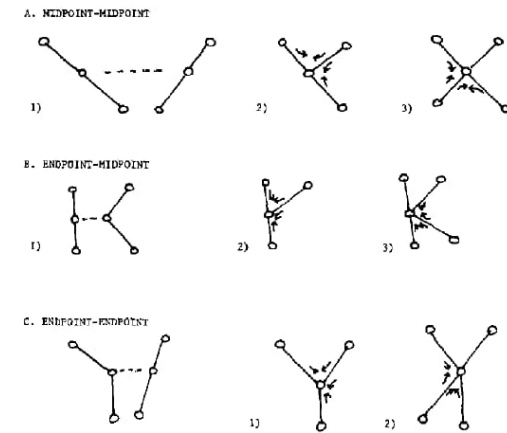


Figure 6-3: merging edges at intersections

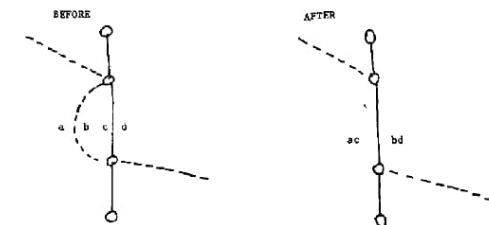


Figure 6-4: coincident edges and history merge process, showing history fields of the edge-sides before and after the history merge process



universität
uulm

5.5 Greiner-Hormann Polygon Clipping

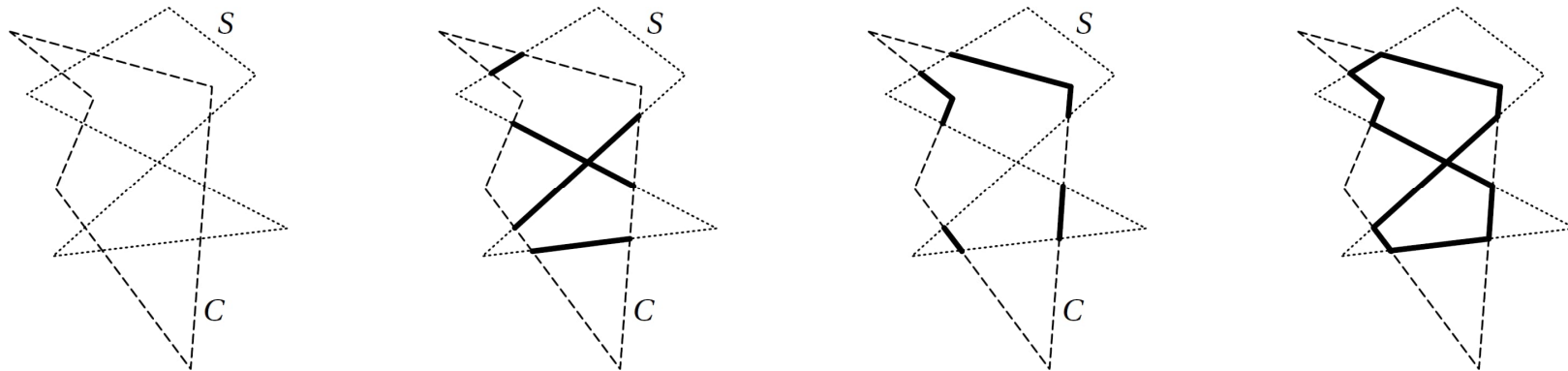
Exploitation of the winding number to clip arbitrary polygons



Visual Computing Group
Institute of Media Informatics

Greiner-Hormann

- Basic Idea: clipped region is made up of all edges of S that lie in C , combined with all edges of C that lie in S



[Greiner & Hormann, TOG 1998]

Chalk Wagon Metaphor

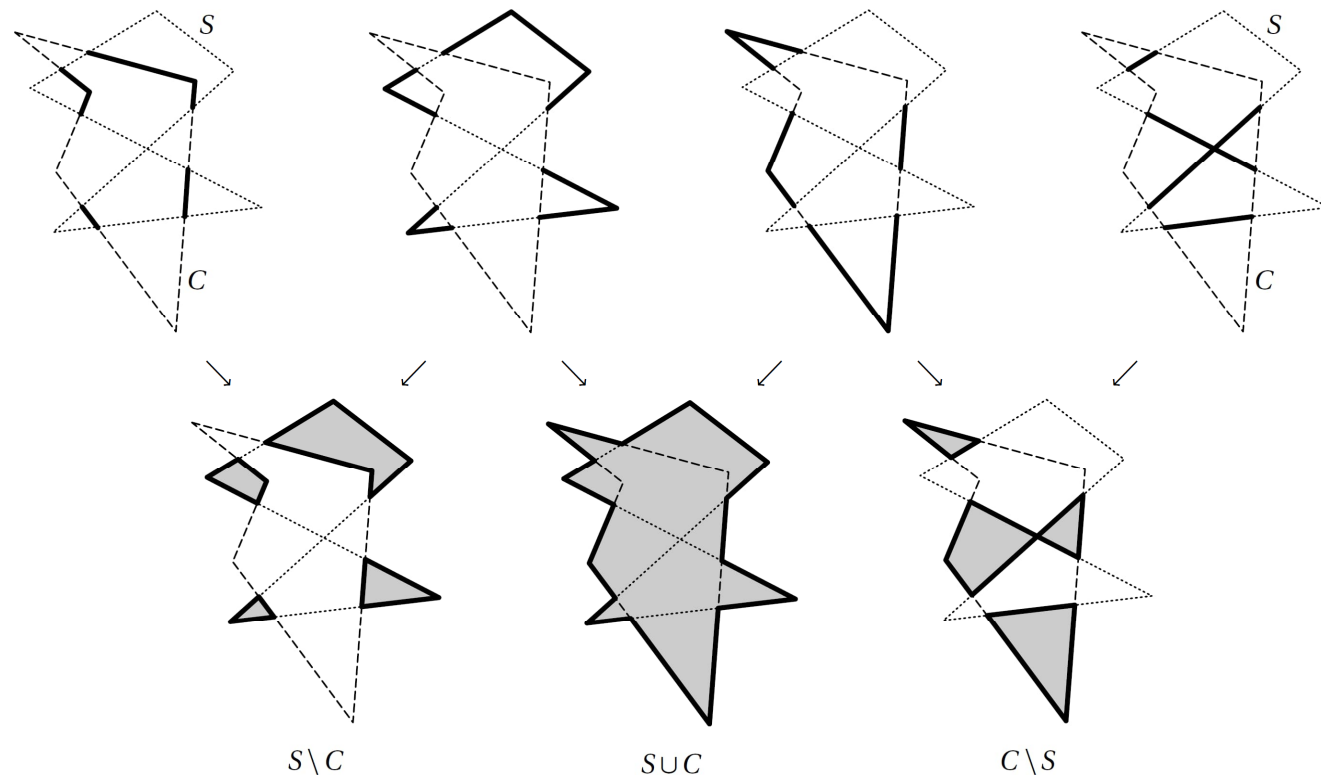
- Using a chalk wagon metaphor
 - Drive along all edges of A with a chalk wagon that is only open when the current position is within B
 - Drive along all edges of B with a chalk wagon that is only open when the current position is within A
 - Join all drawn edges to the clipped polygon



Contour Detection

- The edges of polygon A within polygon B can thus be found as follows
 - Insert in A all intersection points with B
 - Start at any vertex v of A
 - If v is within B , the edge flag is turned on, otherwise not
 - Traverse edges of A and invert edge marker flag when intersecting with B
- Thereafter, the procedure is analogous to the reversed roles of A and B , and the results are merged

Set Operations



[Greiner & Hormann, TOG 1998]

Data Structure 1/2

- Polygons exist as a closed, double-linked list

$$P_0, P_1, P_2, \dots, P_n = P_0$$

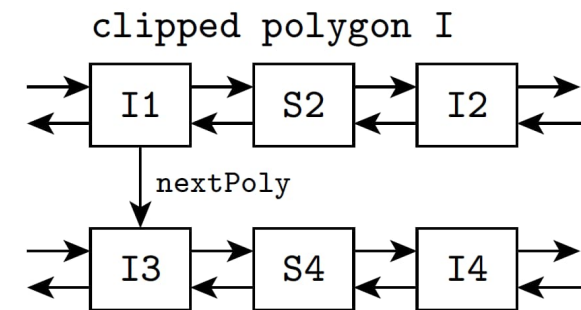
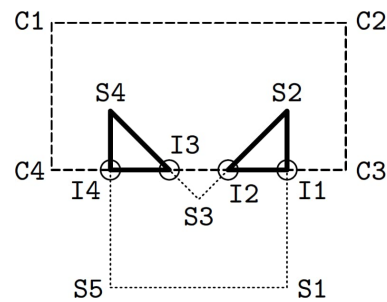
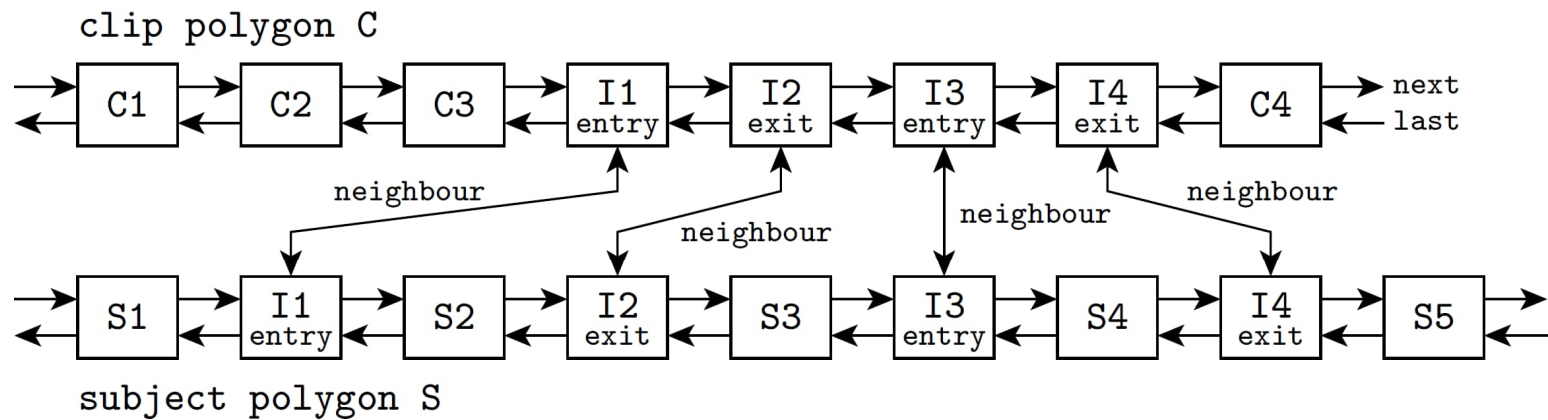
- The following segments are included

$$\overline{P_0P_1}, \overline{P_1P_2}, \dots, \overline{P_{n-1}P_n} = \overline{P_{n-1}P_0}$$

- Used data structure for vertices

```
typedef struct vertex {  
    int x, y;  
    struct vertex *next;  
    struct vertex *prev;  
    struct vertex *nextPoly; // relevant for several unconnected regions  
    bool intersect;          // is current point intersection point?  
    struct vertex *neighbor; // associated intersection point in other polygon  
    bool entry;              // is current point entry point?  
    float alpha;             // interpolation factor of intersection point  
    bool visited;            // has this point been already visited?  
} vertex;
```

Data Structure 2/2

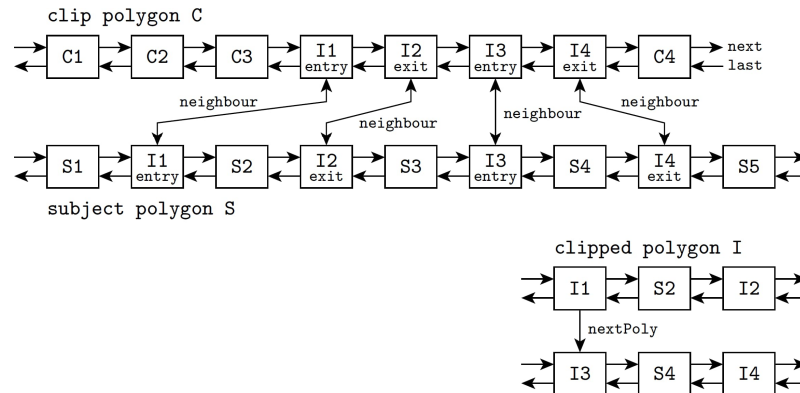
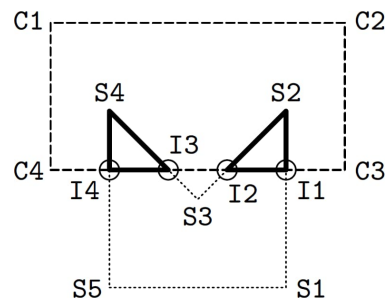


[Greiner & Hormann, TOG 1998]

Proceeding

- Create data structure
 - Create doubly linked list with node data structure
 - Add the intersections with the corresponding alpha values to the list
 - Traverse the polygon to find entry and exit points (use winding number rule)
- Generate the clipping result
 - Traverse data structure and create new polygon with **newPolygon** and **newVertex**

Pseudo Code



```

while unprocessed intersections in subject polygon
  current = first unprocessed intersecting
             point of subject polygon
  newPolygon()
  newVertex(current)
  repeat
    if current->entry
      repeat
        current = current->next
        newVertex(current)
      until current->intersect
    else
      repeat
        current = current->prev
        newVertex(current)
      until current->intersect
    end if
    current = current->neighbor
  until PolygonClosed
end while

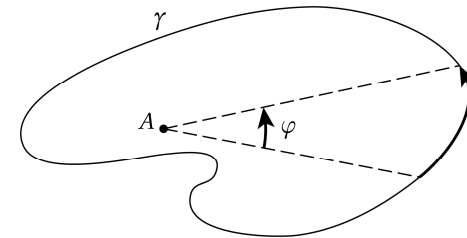
```

[Greiner & Hormann, TOG 1998]

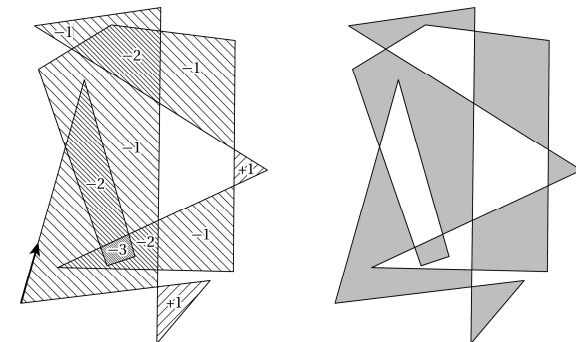
Winding Number 1/3

- Winding number of a point A and a closed curve γ indicates the number of times a ray that traverses the entire curve from A winds around A

$$\omega(\gamma, A) = \frac{1}{2\pi} \int d\varphi$$



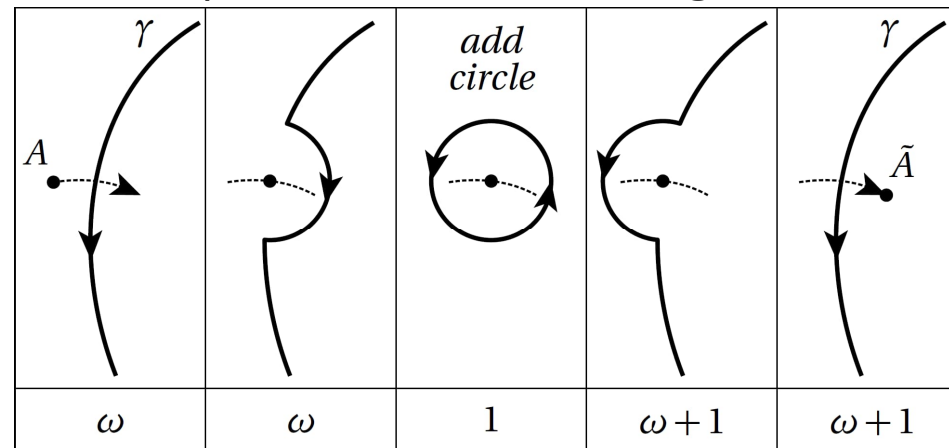
- Counterclockwise: +1
 - Clockwise: -1
- A lies in γ if and only if odd number of winds



[Greiner & Hormann, TOG 1998]

Winding Number 2/3

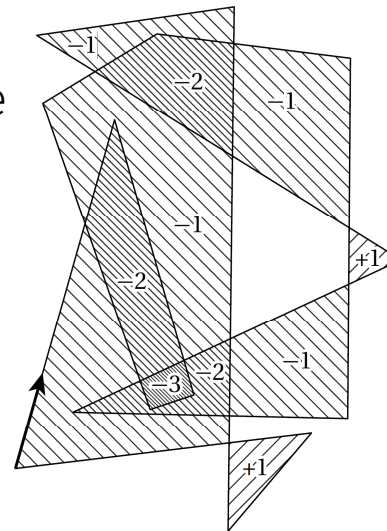
- The winding number of a point A has the following useful properties
 1. If A or the curve γ is continuously changed so that A maintains a positive distance to γ , the winding number will not change
For curve γ , the winding number is constant in each region formed by γ
 2. When A moves and γ crosses once, winding number changes by exactly 1



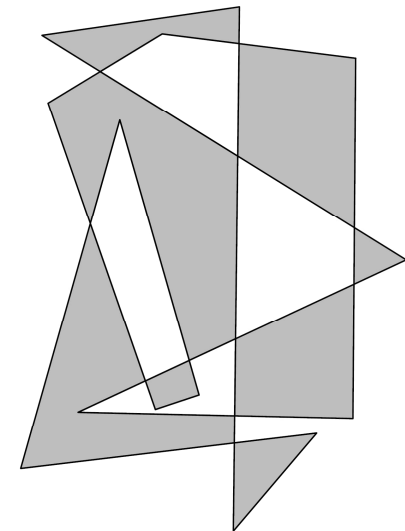
[Greiner & Hormann, TOG 1998]

Winding Number 3/3

- From the second property follows
 - A path that cuts a curve (= the clipping polygon) just once, runs either from inside to outside, or from outside to inside
- Clipping operation can be performed by computing intersections of both polygons
 - Similarly, the union and difference of two polygons can be calculated

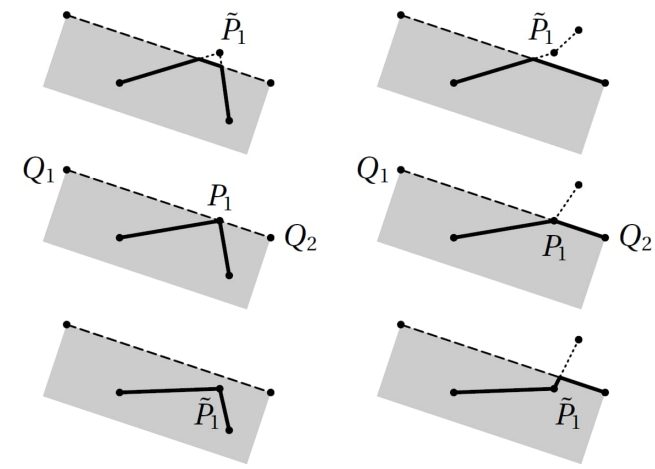


[Greiner & Hormann, TOG 1998]



Special Cases

- Simple cases can be solved after unsuccessful intersection calculation directly via winding number operation
 - One polygon is contained in the other polygon
 - Polygons have no overlaps
- Problem: One point lies on one edge of the other polygon
 - Occurs when an alpha value is 0
 - Apply shift (if possible < pixel size)



failure example 1
with two solutions

failure example 2
with two solutions

[Greiner & Hormann, TOG 1998]

Evaluation

- Allows efficient clipping of arbitrary 2D polygons
 - Self overlaps and holes allowed
- Idea based on definition of winding number
- Very simple and efficient implementation possible
 - Much easier than Weiler-Atherton,
as less special cases have to be considered

Chapter Summary

- Cohen-Sutherland enables **line clipping** against **rectangle** by exploiting binary outcodes
- Cyrus-Beck line clipping enables **line clipping** against **convex polygon** by exploiting parametric representations
- Sutherland-Hodgman polygon clipping enables **polygon** against **convex polygon** clipping by exploiting parametric representations
- Weiler-Atherton polygon clipping enables clipping of **non-convex polygons** against **non-convex polygons**
- Greiner-Hormann polygon clipping enables clipping of **arbitrary polygons** against **arbitrary polygons**



universität
uulm

References

Literature and other sources used in this chapter



Visual Computing Group
Institute of Media Informatics

References

■ Text Books

- J. Foley, A. van Dam, S. Feiner. "Computer Graphics: Principles and Practice (3rd Edition)", Addison-Wesley 2013.
- P. Shirley, M. Ashikhmin, S. Marschner. "Fundamentals of Computer Graphics (4th Edition)", AK Peters 2016.
- B. Sproull, W. M. Newman. "Principles of Interactive Computer Graphics", p. 124, 252, 1973, McGraw-Hill Education, ISBN 0-07-085535-8.

■ Research Publications

- M. Cyrus, J. Beck. "Generalized two-and three-dimensional clipping." Computers & Graphics 3, no. 1 (1978): 23-28.
- G. Greiner, K. Hormann: Efficient Clipping of Arbitrary Polygons, ACM Transactions on Graphics, 17(2):71-83, 1998.
- I. E. Sutherland, G. W. Hodgman. "Reentrant polygon clipping." Communications of the ACM 17, no. 1 (1974): 32-42.
- K. Weiler, P. Atherton. "Hidden surface removal using polygon area sorting." In ACM SIGGRAPH computer graphics, vol. 11, no. 2, pp. 214-222. ACM, 1977.